# Week 6

# Algorithm Efficiency

## Chapter 10

© 2017 Pearson Education, Hoboken, NJ.  All rights reserved

# What Is a Good Solution?

- A program incurs a real and tangible cost.
  - Computing time
  - Memory required
  - Difficulties encountered by users
  - Consequences of incorrect actions by program
- A solution is good if …
  - The total cost incurs …
  - Over all phases of its life … is minimal

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# What Is a Good Solution?

- Important elements of the solution
  - Good structure
  - Good documentation
  - Efficiency
- Be concerned with efficiency when
  - Developing underlying algorithm
  - Choice of objects and design of interaction between those objects

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Measuring Efficiency of Algorithms

- Important because
  - Choice of algorithm has significant impact
- Examples
  - Responsive word processors
  - Grocery checkout systems
  - Automatic teller machines
  - Video machines
  - Life support systems

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Measuring Efficiency of Algorithms

- Analysis of algorithms
  - The area of computer science that provides tools for contrasting efficiency of different algorithms
  - Comparison of algorithms should focus on significant differences in efficiency
  - We consider comparisons of *algorithms*, not programs

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Measuring Efficiency of Algorithms

- Difficulties with comparing programs (instead of algorithms)
  - How are the algorithms coded
  - What computer will be used
  - What data should the program use
- Algorithm analysis should be independent of
  - Specific implementations, computers, and data

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# The Execution Time of Algorithms

- An algorithm's execution time is related to number of operations it requires.
  - Algorithm's execution time is related to number of operations it requires.
- Example: Towers of Hanoi
  - Solution for *n* disks required $2^n - 1$ moves
  - If each move requires time *m*
  - Solution requires $(2^n - 1) \times m$ time units

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Execution Time Evolution

- In RDBMS, access times are critical
  - RAM is in nanoseconds
  - In 1980s-1990s, hard disk access times ~20 millis
  - Modern SSDs access times ~9 microseconds
- Query optimizers had to be rewritten
  - All access times are now parameterized/metadata
  - No more hardcoding of values

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Algorithm Growth Rates

- Measure an algorithm's time requirement as function of problem size
- Most important thing to learn
  - How quickly algorithm's time requirement grows as a function of problem size

*Algorithm A requires time proportional to* $n^2$
*Algorithm B requires time proportional to* $n$

  - Demonstrates contrast in growth rates

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Algorithm Growth Rates

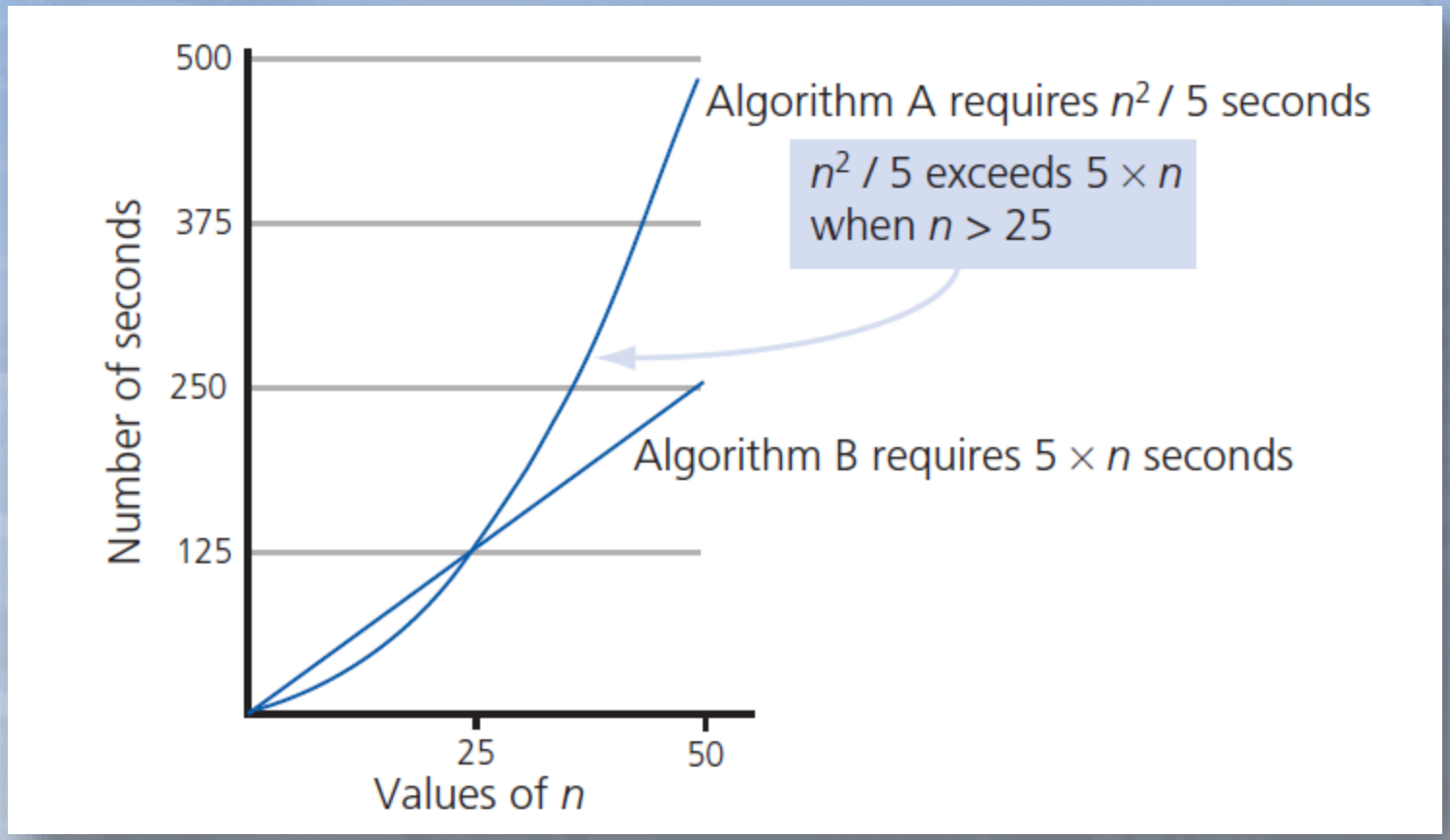


FIGURE 10-1 Time requirements as a function of problem size *n*

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Analysis and Big O Notation

- Algorithm *A* is said to be order $f(n)$,
  - Denoted as $O(f(n))$
  - Function f(n) called algorithm's growth rate function
  - Notation with capital O denotes *order*
- Algorithm *A* of order denoted O(f(n))
  - Constants $k$ and $n_0$ exist such that
  - *A* requires no more than $k \times f(n)$ time units
  - For problem of size $n \geq n_0$

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Analysis and Big O Notation

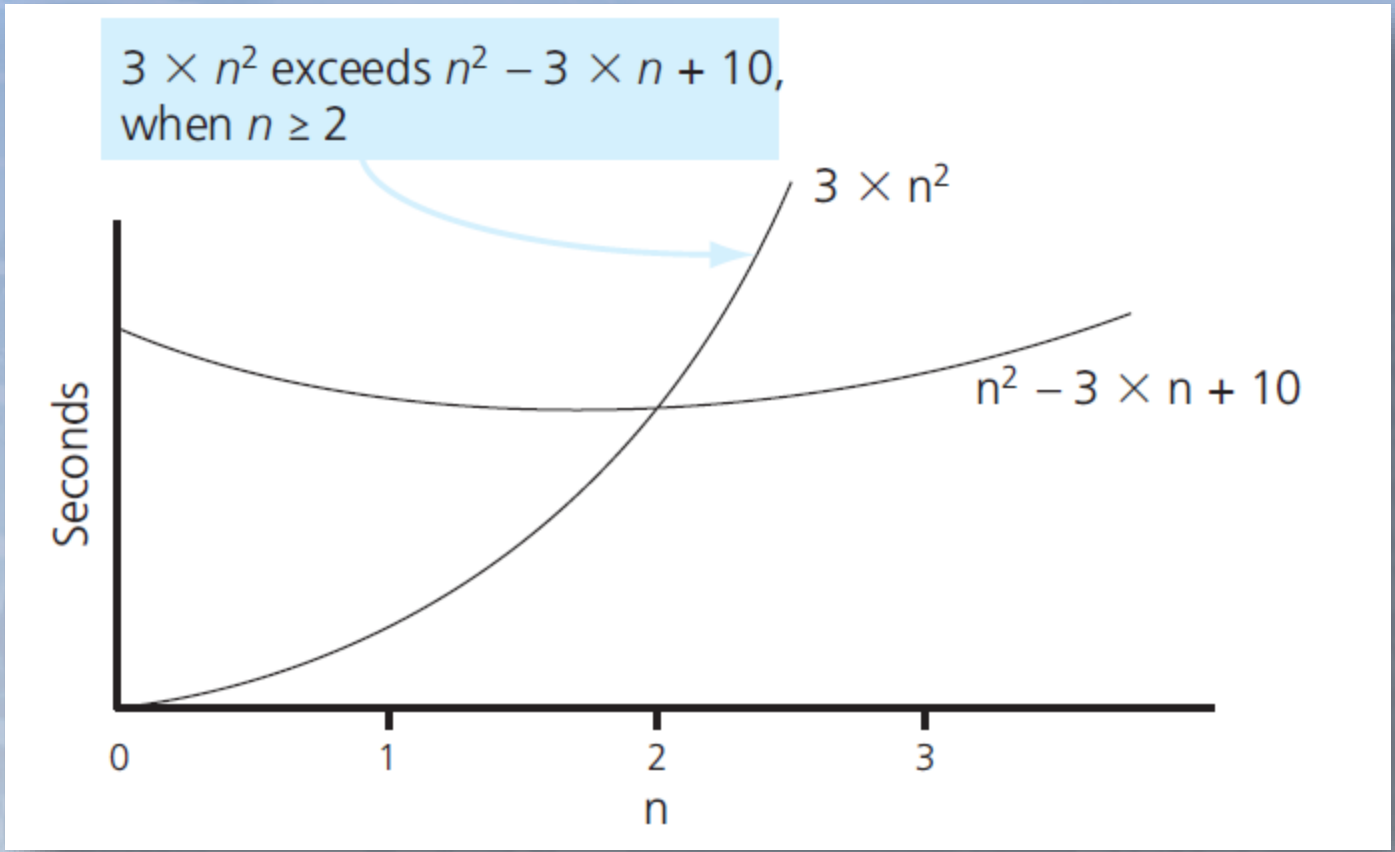


FIGURE 10-2 The graphs of $3 \times n^2$ and $n^2 - 3 \times n + 10$

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Analysis and Big O Notation

$$O(1) < O(\log_2 n) < O(n) < O(n \times \log_2 n) <$$

$$) < O(n^2) < O(n^3) < O(2^n)$$

Order of growth of some common functions

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Analysis and Big O Notation

| Function | n = 10 | 100 | 1,000 | 10,000 | 100,000 | 1,000,000 |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| $\log_2 n$ | 3 | 6 | 9 | 13 | 16 | 19 |
| $n$ | 10 | $10^2$ | $10^3$ | $10^4$ | $10^5$ | $10^6$ |
| $n \times \log_2 n$ | 30 | 664 | 9,965 | $10^5$ | $10^6$ | $10^7$ |
| $n^2$ | $10^2$ | $10^4$ | $10^6$ | $10^8$ | $10^{10}$ | $10^{12}$ |
| $n^3$ | $10^3$ | $10^6$ | $10^9$ | $10^{12}$ | $10^{15}$ | $10^{18}$ |
| $2^n$ | $10^3$ | $10^{30}$ | $10^{301}$ | $10^{3,010}$ | $10^{30,103}$ | $10^{301,030}$ |

FIGURE 10-3 A comparison of growth-rate functions

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Analysis and Big O Notation

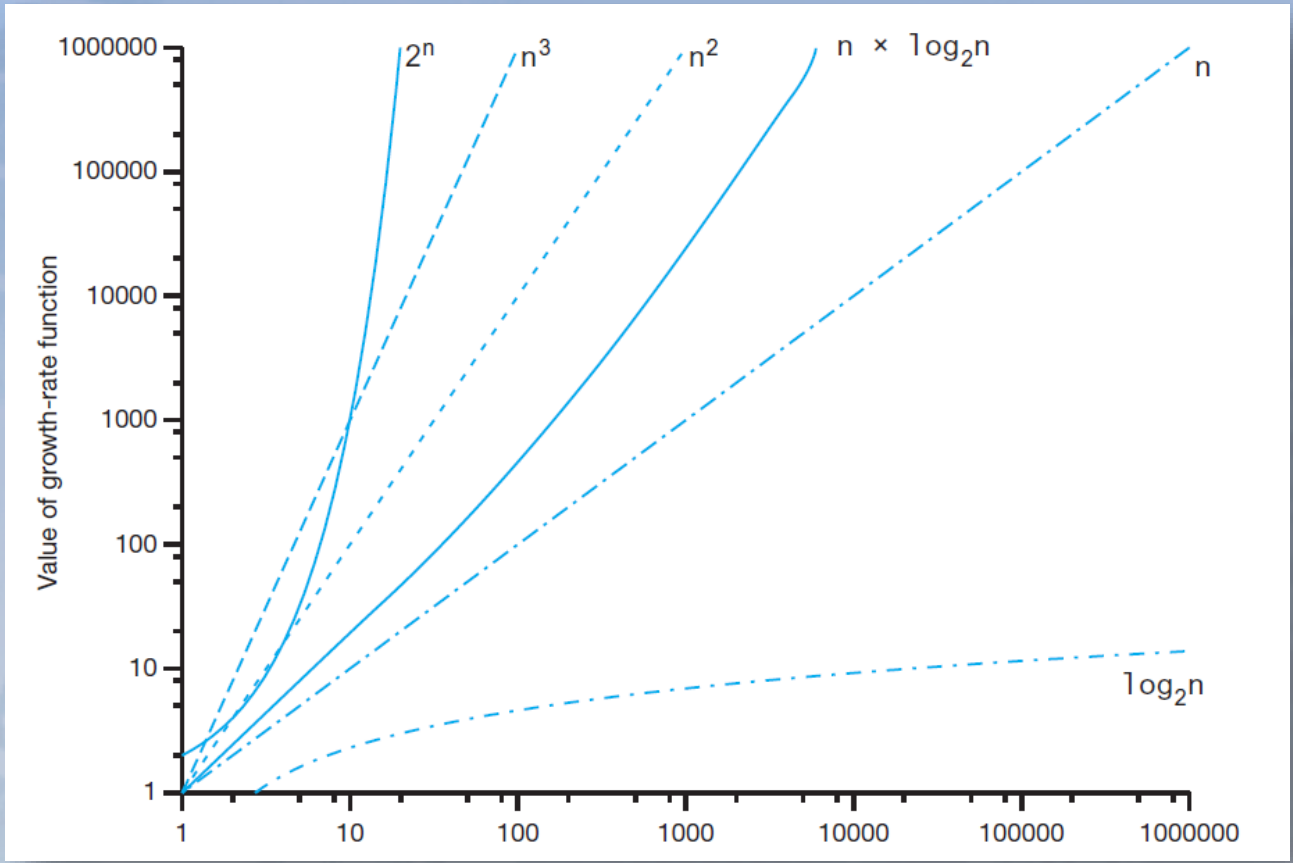


FIGURE 10-4 A comparison of growth-rate functions

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# RDBMS - Small Amounts of Data

- In RDBMS, indexes are used for performance
  - Indexes are Balanced Trees stored on disk
  - Presorted data structures
- For large amounts of data, indexes are critical
- For small amounts of data, indexes are not needed
  - MySQL default page size is 16K (can be changed)
  - Data under 2-3 dozen pages is “small”

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Analysis and Big O Notation

- Worst-case analysis
  - Worst case analysis usually considered
  - Easier to calculate, thus more common
- Average-case analysis
  - More difficult to perform
  - Must determine relative probabilities of encountering problems of given size

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Keeping Your Perspective

- ADT used makes a difference
  - Array-based getEntry is O(1)
  - Link-based getEntry is O(n)
- Choosing implementation of ADT
  - Consider how frequently certain operations will occur
  - Seldom used but critical operations must also be efficient

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Keeping Your Perspective

- If problem size is always small
  - Possible to ignore algorithm's efficiency
- Weigh trade-offs between
  - Algorithm's time and memory requirements
- Compare algorithms for style and efficiency

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Efficiency of Searching Algorithms

- Sequential search
  - Worst case: O($n$)
  - Average case: O($n$)
  - Best case: O(1)
- Binary search
  - O($\log_2 n$) in worst case
  - At same time, maintaining array in sorted order requires overhead cost … can be substantial

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# End

Chapter 10

© 2017 Pearson Education, Hoboken, NJ.  All rights reserved